



Sunbeam Institute of Information Technology
Pune and Karad

Data structures and Algorithms

Trainer - Devendra Dhande

Email – devendra.dhande@sunbeaminfo.com

Insertion sort

1. Pick one element of the array (start from 2nd element)
2. Compare picked element with all its left neighbours one by one
3. If left neighbour is greater, move it one position ahead
4. Insert picked element at its appropriate position
5. Repeat above steps until array is sorted (N-1)

pass 1: 11 22 33 hh 55 66
 pass 2: 11 22 33 hh 55 66
 pass 3: 11 22 33 hh 55 66
 pass 4: 11 22 33 hh 55 66
 pass 5: 11 22 33 hh 55 66

No. of comps = $n-1$

Time $\propto n-1$

$T(n) = O(n)$ ← Best

n - No. of elements	
Passes	comps
1	1
2	2
3	3
$\vdots$	$\vdots$
$n-1$	$n-1$

$$\text{Total comps} = 1 + 2 + 3 + \dots + (n-2) + (n-1)$$

$$= \frac{n(n-1)}{2}$$

$$\text{Time} \propto \frac{1}{2} (n^2 - n)$$

$$T(n) = O(n^2)$$

← Avg
 ← Worst

Processing variables: i, j, temp (constant space)

$$S(n) = O(1)$$

Insertion sort

50	40	20	60	10	30
0	1	2	3	4	5

40
temp

20
temp

60
temp

10
temp

30
temp

50		20	60	10	30
0	1	2	3	4	5

	50	20	60	10	30
0	1	2	3	4	5

40	50	20	60	10	30
0	1	2	3	4	5

40	50		60	10	30
0	1	2	3	4	5

40		50	60	10	30
0	1	2	3	4	5

	40	50	60	10	30
0	1	2	3	4	5

20	40	50	60	10	30
0	1	2	3	4	5

20	40	50		10	30
0	1	2	3	4	5

20	40	50	60	10	30
0	1	2	3	4	5

20	40	50	60		30
0	1	2	3	4	5

20	40	50		60	30
0	1	2	3	4	5

20	40		50	60	30
0	1	2	3	4	5

20		40	50	60	30
0	1	2	3	4	5

	20	40	50	60	30
0	1	2	3	4	5

10	20	40	50	60	30
0	1	2	3	4	5

10	20	40	50	60	
0	1	2	3	4	5

10	20	40	50		60
0	1	2	3	4	5

10	20	40		50	60
0	1	2	3	4	5

10	20		40	50	60
0	1	2	3	4	5

10	20	30	40	50	60
0	1	2	3	4	5

to pick elements : $i = 1 \rightarrow N-1$ ($i < N$) $i++$
 to compare on left side : $j = i-1 \rightarrow 0$ ($j \geq 0$) $j--$

```

for( i=1 ; i < N ; i++ ) {
    temp = arr[i];
    int j;
    for( j=i-1 ; j >= 0 ; j-- ) {
        if ( arr[j] > temp )
            arr[j+1] = arr[j];
        else
            break;
    }
    arr[j+1] = temp;
}

```

10	20	30	40	50	60
0	1	2	3	4	5

i	i < 6	temp	j	insert pos
1	T	40	0, -1	0
			✓ ✗	
2	T	20	1, 0, -1	0
			✓ ✓ ✗	
3	T	60	2	3
			✓	
4	T	10	3, 2, 1, 0, -1	0
			✓ ✓ ✓ ✓ ✗	
5	T	30	4, 3, 2, 1	2
			✓ ✓ ✓ ✓	

Stable Vs Unstable

Before sort : 22 11 33 11

After sort : 11 11 22 33 ← stable (bubble, insertion, merge)

After sort : 11 11 22 33 ← unstable (selection, quick, heap)

In place Vs Out place

in place → sorting is done within array space
bubble, insertion, selection, quick, heap

out place → sorting is done out of array space (temp array)
merge

Missing Number

Given an array nums containing n distinct numbers in the range [0, n], return the only number in the range that is missing from the array.

Example 1:

Input: nums = [3,0,1]

Output: 2

$$nSum = 0 + 1 + 2 + 3 = 6$$

$$arraySum = 3 + 0 + 1 = 4$$

$$\begin{aligned} \text{missing number} &= nSum - arraySum \\ &= 6 - 4 \\ &= 2 \end{aligned}$$

Example 2:

Input: nums = [0,1]

Output: 2

Example 3:

Input: nums = [9,6,4,2,3,5,7,0,1]

Output: 8

- ① find sum of n numbers
- ② find sum of array element
- ③ find diff of both $\leftarrow$ missing number

```
int missingNumber(int nums[]) {  
    int n = nums.length;  
    int nSum = n * (n + 1) / 2;  
    int numsSum = 0;  
    for (int i = 0; i < n; i++)  
        numsSum += nums[i];  
    return nSum - numsSum;  
}
```

$$T(n) = O(n)$$

Find smallest letter greater than target

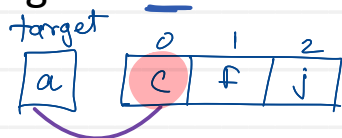
You are given an array of characters letters that is sorted in non-decreasing order, and a character target. There are at least two different characters in letters.

Return the smallest character in letters that is lexicographically greater than target. If such a character does not exist, return the first character in letters.

Example 1:

Input: letters = ["c", "f", "j"], target = "a"

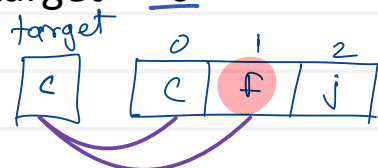
Output: "c"



Example 2:

Input: letters = ["c", "f", "j"], target = "c"

Output: "f"



Example 3:

Input: letters = ["x", "x", "y", "y"], target = "z"

Output: "x"

```
char nextGreatestLetter(char[] letters, char target)
{
    int n = letters.length;
    for(int i = 0; i < n; i++) {
        if(target < letters[i])
            return letters[i];
    }
    return letters[0];
}
```

$$T(n) = O(n)$$

Remove Duplicates from sorted array

Given an integer array nums sorted in non-decreasing order, remove the duplicates in-place such that each unique element appears only once. The relative order of the elements should be kept the same. Then return the number of unique elements in nums.

Example 1:

Input: nums = [1,1,2]

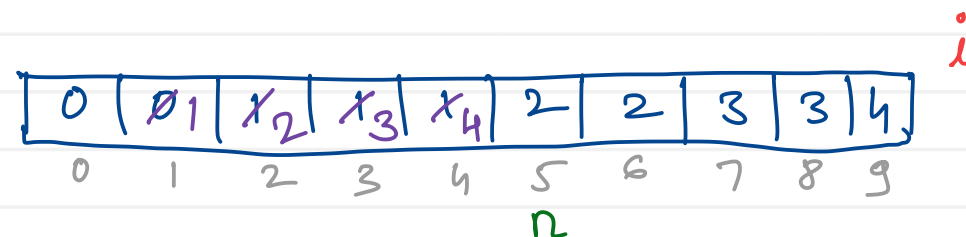
Output: 2, nums = [1,2,_]

Example 2:

Input: nums = [0,0,1,1,1,2,2,3,3,4]

Output: 5, nums = [0,1,2,3,4,_,_,_,_,_]

Two pointers technique



```
int removeDuplicates(int[] nums) {
    int s = nums.length;
    int n = 1;
    for (int i = 1; i < n; i++) {
        if (nums[i] != nums[n-1])
            nums[n++] = nums[i];
    }
    return n;
}
```

3

Search insert position

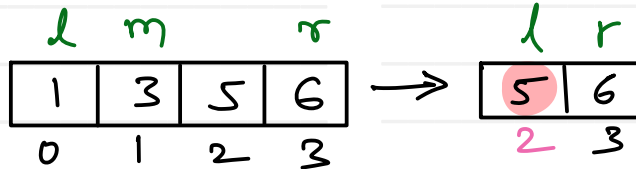
Given a sorted array of distinct integers and a target value, return the index if the target is found. If not, return the index where it would be if it were inserted in order.

You must write an algorithm with $O(\log n)$ runtime complexity.

Example 1:

Input: nums = [1,3,5,6], target = 5

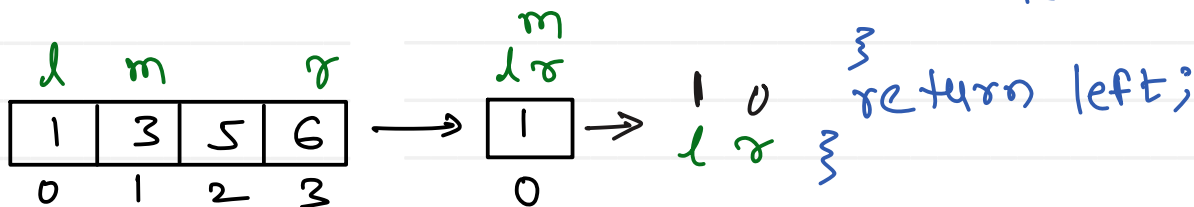
Output: 2



Example 2:

Input: nums = [1,3,5,6], target = 2

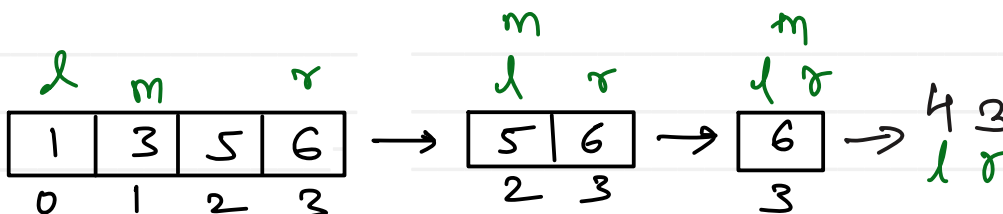
Output: 1



Example 3:

Input: nums = [1,3,5,6], target = 7

Output: 4



```
int searchInsert(int nums[], int target) {
    int left = 0, right = nums.length - 1, mid;
    while (left <= right) {
        mid = (left + right) / 2;
        if (target == nums[mid])
            return mid;
        if (target < nums[mid])
            right = mid - 1;
        else
            left = mid + 1;
    }
    return left;
}
```

Find first and last position of element in sorted array

Given an array of integers nums sorted in non-decreasing order, find the starting and ending position of a given target value.

If target is not found in the array, return [-1, -1].

You must write an algorithm with $O(\log n)$ runtime complexity.

Example 1:

Input: nums = [5,7,7,8,8,10], target = 8

Output: [3,4]

Example 2:

Input: nums = [5,7,7,8,8,10], target = 6

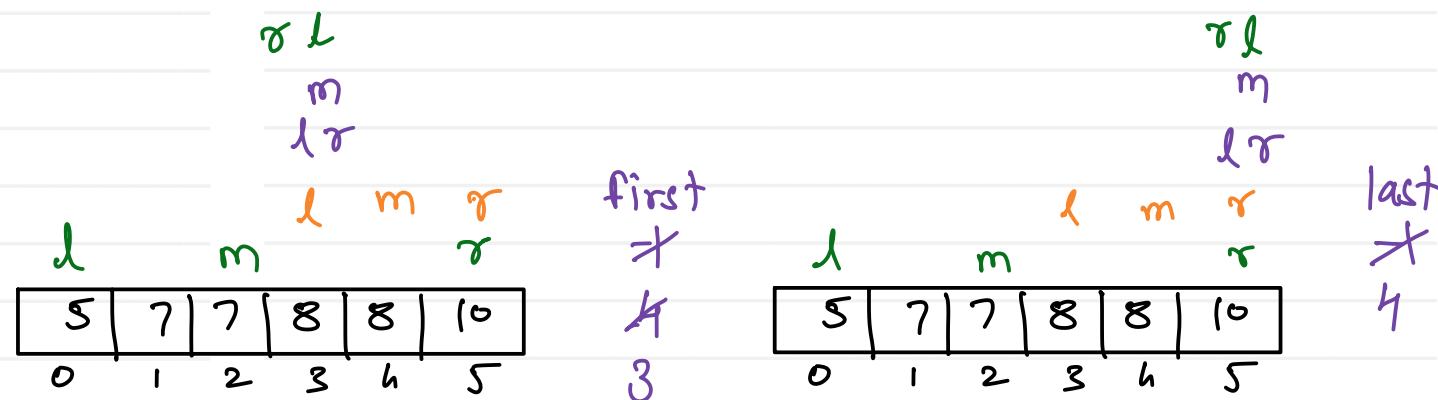
Output: [-1,-1]

Example 3:

Input: nums = [], target = 0

Output: [-1,-1]

1. Search target into the array
2. if key is found, find first position of target
3. if key is not found, return [-1, -1]
4. if key is found, find last position of target



find first position

```
left = 0, right = nums.length - 1
first = -1;
while (left <= right) {
    mid = (left + right) / 2;
    if (target == nums[mid]) {
        first = mid;
        right = mid - 1;
    } else if (target < nums[mid]) {
        right = mid - 1;
    } else {
        left = mid + 1;
    }
}
```

find last position

```
left = 0, right = nums.length - 1
last = -1;
while (left <= right) {
    mid = (left + right) / 2;
    if (target == nums[mid]) {
        last = mid;
        left = mid + 1;
    } else if (target < nums[mid]) {
        right = mid - 1;
    } else {
        left = mid + 1;
    }
}
```

Search in rotated sorted array

There is an integer array nums sorted in ascending order (with distinct values).

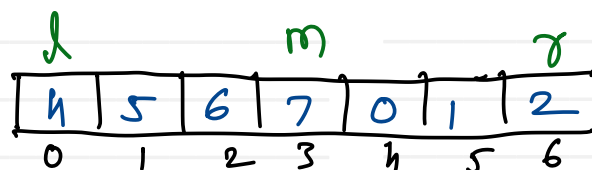
Prior to being passed to your function, nums is possibly rotated at an unknown pivot index k ($1 \leq k < \text{nums.length}$) For example, [0,1,2,4,5,6,7] might be rotated at pivot index 3 and become [4,5,6,7,0,1,2].

Given the array nums after the possible rotation and an integer target, return the index of target if it is in nums, or -1 if it is not in nums. You must write an algorithm with $O(\log n)$ runtime complexity.

Example 1:

Input: $\text{nums} = [4,5,6,7,0,1,2]$, $\text{target} = 0$

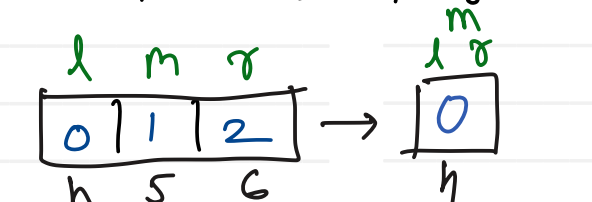
Output: 4



Example 2:

Input: $\text{nums} = [4,5,6,7,0,1,2]$, $\text{target} = 3$

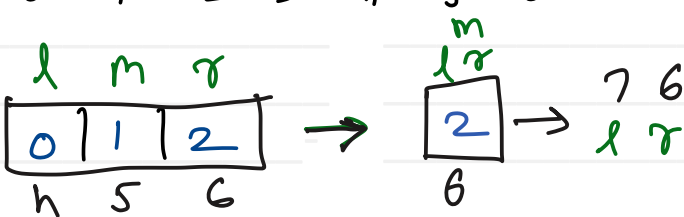
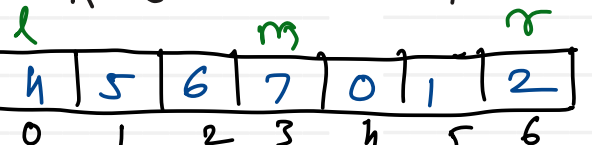
Output: -1



Example 3:

Input: $\text{nums} = [1]$, $\text{target} = 0$

Output: -1



```

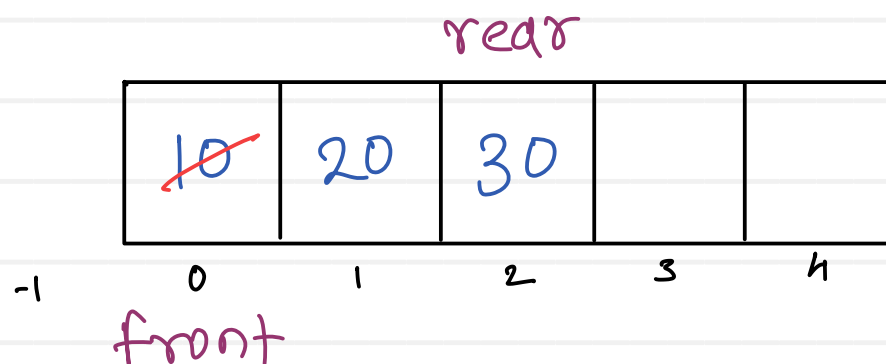
l = 0, r = nums.length - 1, m;
while (l <= r) {
    m = (l + r) / 2;
    if (target == nums[m])
        return m;
    if (nums[l] <= nums[m]) {
        if (nums[l] <= target && target < nums[m])
            right = mid - 1;
        else
            left = mid + 1;
    } else {
        if (nums[m] < target && target <= nums[r])
            left = mid + 1;
        else
            right = mid - 1;
    }
}

```

Linear queue

- linear data structure which has two ends - front and rear
- Data is inserted from rear end and removed from front end
- Queue works on the principle of “First In First Out” / “FIFO”

Capacity = 5



Operations :

1. Insert / Enqueue / Push / offer :

- a. reposition rear (++)
- b. add value at rear index

2. Remove / Dequeue / Pop / poll :

- a. reposition front (++)

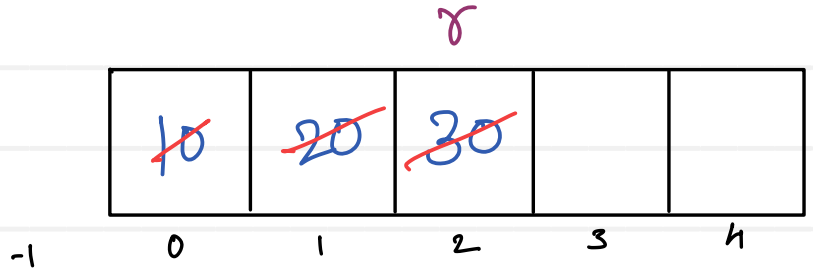
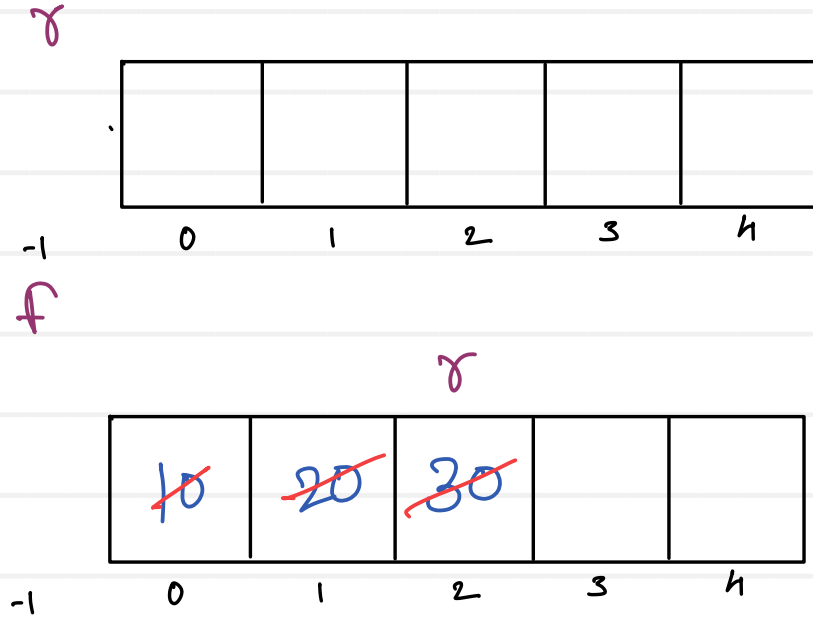
3. Peek :

- a. read value from front end (front+1)

- All operations will be performed in $O(1)$ time complexity.

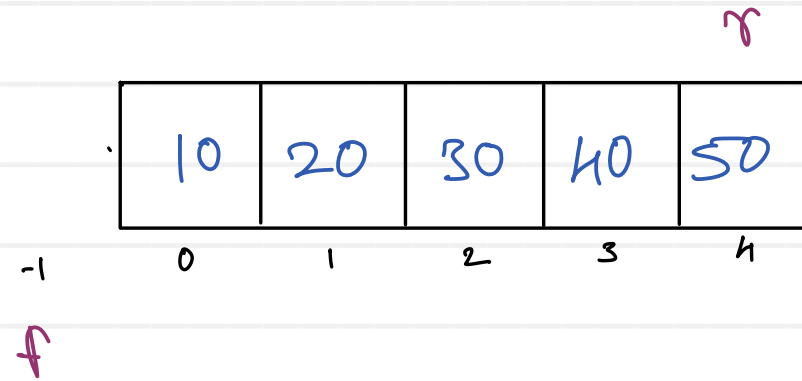
Linear queue - Conditions

Empty



$front == rear$

Full



$rear == size - 1$